StudyForge

Stage 2 Implementation Report

EECS3311 Software Design / Fall 2026 / Prepared November 12, 2026, Toronto local time.

1 Project overview and implementation status

StudyForge helps a university student turn course materials into evidence-linked explanations, practice and scheduled study actions. A local browser GUI and CLI share the same controller, commands, SQLite records and feature services. Source code now implements a path for each of the twelve original Stage 1 features and seven distinct design patterns. The original main branch contained only the Stage 1 report and submission link; there was no earlier runnable application.

The production adapter calls OpenAI Responses with the configured Stage 1 model, gpt-4.1-mini-2025-04-14. Retrieval and recent conversation memory bound the context; strategies validate output schemas and citation identities before saving artifacts. The focused-session orchestrator selects registered explanation and practice tools, reads current progress to choose practice, and checkpoints results and legal state transitions. This is a multi-step agent workflow rather than a raw prompt-forwarding interface. Deterministic policies control grading, dates, workload, approvals, calendar conflicts, review intervals and learning metrics.

Verification status: 39 automated tests passed, including complete CLI and local HTTP API journeys through all twelve features with real SQLite persistence and reopening. Only the remote model is replaced by a test double in those journeys. The GUI was manually inspected for course creation, real material import and safe missing-key errors. A stale-output defect discovered in that check was corrected. No live provider request was made: a local API key has not been supplied.

Submission is not complete. The owner will configure API access, validate actual generated output, record and upload all feature demonstrations, add accessible timestamped links, supply measured AI usage/cost accounting and submit the course repository URL. Historical weekly activity is absent in the original repository; current commits do not establish development or pushes in the required past periods. The original November 10 deadline has passed. No grade or on-time status is claimed.

Scope changes are explicit: lexical retrieval replaces embeddings; browser GUI replaces a native window; local ICS export replaces remote calendar writes; basic descriptive analytics and bounded sessions replace broader planned behavior. Sections 2-4 account for every original feature and affected class/pattern.

2 Feature to implementation mapping

All feature IDs refer to the original Stage 1 feature with the same ID and UC number. Each has an implemented path, with modifications listed below and in architecture.md. Automated acceptance uses a model test double; live provider acceptance is pending. Every video entry is awaiting owner recording, as indexed in demo-index.json.

F01 Material ingestion and indexing / UC01

Status: Modified; deterministic implementation.

Files/classes: learning.py DocumentIngestTool extracts and chunks supported files; RetrievalTool ranks saved chunks; core.py SQLiteRepository persists material records; EventBus publishes MaterialIndexed to ProgressProjector.

Methods: DocumentIngestTool.execute(), _extract(); RetrievalTool.execute(); SQLiteRepository.save(), transaction(); EventBus.publish().

Collaboration: GUI or CLI sends import_materials through StudyController and StudyCommand. The tool validates each path and 20 MiB limit, hashes bytes, extracts text, creates overlapping chunks with stable locations, saves, and emits an event in a per-file transaction. One rejected file does not discard other successes.

Patterns: MVC, Command, Adapter (format parsing), Observer. Changes: filename-derived metadata and lexical index replace AI tagging and vector embeddings. OCR is absent. Video: F01 segment pending; show valid file, duplicate, unsupported file and sources.

F02 Grounded summaries / UC02

Status: Modified; hybrid.

Files/classes: learning.py LearningService coordinates RetrievalTool and SummaryStrategy; CitationVerifier checks IDs; model.py OpenAIModelAdapter performs generation; core.py SQLiteRepository saves summary and evidence.

Methods: LearningService.summarize(), _generate(); GenerationStrategy.generate(); SummaryStrategy.validate(); CitationVerifier.verify(); ModelGateway.generate().

Collaboration: material/detail/focus input selects scoped chunks, constructs the strategy contract, calls the provider, validates nonempty text and citations, retries malformed output once, then saves the result and source trail. No evidence produces an error before generation. Coverage reports bounded retrieval explicitly.

Patterns: Strategy, Adapter, Command, MVC. Changes: no section-domain selector or exhaustive whole-course summary; source-ID validity does not prove semantic entailment. Video: F02 pending; show detail/focus, source resolution and missing evidence behavior.

F03 Concept extraction and knowledge map / UC03

Status: Modified; hybrid.

Files/classes: learning.py ConceptGraphStrategy validates and normalizes nodes/edges; LearningService retrieves and persists; web/app.js renders the SVG graph and evidence.

Methods: LearningService.concepts(); ConceptGraphStrategy.validate(); CitationVerifier.verify(); browser renderGraph().

Collaboration: selected course/material/topic supplies evidence to the model; names are merged case-insensitively, confidence values and edge endpoints/types are checked, and the saved graph is rendered with relationships and source records.

Patterns: Strategy, Adapter, MVC. Changes: invalid graph rejects and retries rather than quarantining individual edges; regeneration creates a new record without a version comparison service. Video: F03 pending; show graph and supporting evidence.

F04 Grounded question answering / UC04

Status: Modified; hybrid.

Files/classes: LearningService, RetrievalTool, GroundedAnswerStrategy, CitationVerifier (learning.py); OpenAIModelAdapter (model.py); SQLiteRepository.

Methods: ask(); RetrievalTool.execute(); GenerationStrategy.generate(); GroundedAnswerStrategy.validate(); SQLiteRepository.list(), save().

Collaboration: the controller validates course scope, retrieval ranks only that course's chunks, and the model receives current question and the latest four saved conversation turns. Verified text/citations/confidence and evidence are saved. Lexical misses return an evidence-gap error, without inventing an answer.

Patterns: Strategy, Facade, Command, Adapter, MVC. Changes: explicit commands replace IntentRouter and multi-query TaskPlanner; pronoun-only questions may need rephrasing. Video: F04 pending; show answer/citations, follow-up and unrelated question error.

F05 Personalized study plan / UC05

Status: Modified; hybrid.

Files/classes: core.py StudyAgentFacade handles plan generation, feasibility and approval; ProgressService supplies mastery; model.py OpenAIModelAdapter orders topics.

Methods: plan_create(); plan_approve(); get_snapshot(); SQLiteRepository.transaction().

Collaboration: goal and mastery go to the planner model; deadline and daily availability are enforced locally. Scoped course evidence is used even for broad goals. Deterministic rules validate topic uniqueness, known graph prerequisite order, deadline and slot capacity; dated sessions are saved as a draft. Approval is separate; active replacement is explicit.

Patterns: Facade, Command, Adapter, MVC. Changes: daily-minute capacity replaces arbitrary availability windows/profile priorities; activities and checkpoints are not richly generated. Video: F05 pending; show draft, approval, infeasible capacity and replacement guard.

F06 Flashcard generation and review / UC06

Status: Modified; hybrid generation with deterministic review.

Files/classes: FlashcardStrategy and LearningService (learning.py) generate, edit, reject and queue cards; ReviewScheduler and StudyAgentFacade (core.py) record reviews; EventBus and ProgressProjector preserve activity.

Methods: flashcards(); card_update(); due_queue(); review(); schedule_next().

Collaboration: topic/count/difficulty retrieve evidence and generate unique cited cards. Reviewing a card with rating 0-3 saves its next interval/due date and publishes CardReviewed. Queue excludes rejected/not-yet-due cards. Edits are marked user_edited.

Patterns: Strategy, Command, Observer, Adapter, MVC. Changes: simple documented rating multiplier; user edits are not AI-reverified. Video: F06 pending; show deck, review, due schedule and edit/reject alternatives.

F07 Quiz generation / UC07

Status: Modified; hybrid.

Files/classes: learning.py QuizStrategy supplies generation and validation contract; LearningService retrieves and stores quiz; ModelGateway supplies generation.

Methods: quiz(); QuizStrategy.validate(); GenerationStrategy.generate().

Collaboration: the user chooses topic/count/difficulty; model creates choice/free questions. Rules require unique questions, count, valid objective options/keys, free-response rubrics and resolvable citations. Saved quiz includes answer key and evidence. Invalid output retries once, then fails without a partial saved quiz.

Patterns: Strategy, Command, Adapter, MVC. Changes: no explicit objective coverage validator, user-selected format mix or partial-quiz fallback. Video: F07 pending; show saved questions/rubrics/citations and invalid count.

F08 Automatic grading and explanations / UC08

Status: Modified; hybrid.

Files/classes: core.py AssessmentService grades; OpenAIModelAdapter evaluates free text against a rubric; SQLiteRepository stores attempt/rubric snapshot; EventBus and ProgressProjector record accepted activity.

Methods: grade(); ModelGateway.generate(); EventBus.publish(); get_snapshot().

Collaboration: all item IDs must have nonempty answers. Exact normalized option text is scored deterministically; free answers receive model score, feedback and confidence. Total, per-item citations, rubric and evidence are persisted atomically. Low-confidence item scores are flagged and excluded from mastery.

Patterns: Command, Adapter, Observer, Facade. Changes: no human grade-override UI; duplicate protection uses a supplied stable attempt ID, while new IDs denote new attempts. Video: F08 pending; show mixed scoring, feedback, incomplete submission and duplicate ID rejection.

F09 Weak-topic diagnosis and adaptive practice / UC09

Status: Modified; hybrid with deterministic recommendation.

Files/classes: core.py ProgressService and StudyAgentFacade; learning.py QuizStrategy.

Methods: get_snapshot(); adapt(); LearningService.quiz().

Collaboration: accepted evidence is aggregated. At least two observations and mastery below 0.7 are required; the lowest topic is proposed with its sample count and an introductory/intermediate difficulty. Confirmation generates bounded practice with that actual difficulty, saving a quiz and adaptation record.

Patterns: Facade, Command, Strategy (reused quiz generator), Adapter. Changes: no prerequisite remediation traversal, AI narrative or insertion into active plan. Sparse evidence is reported honestly. Video: F09 pending; show insufficient evidence, then weak-topic proposal and confirmed practice.

F10 Progress and mastery analytics / UC10

Status: Modified; deterministic.

Files/classes: core.py ProgressProjector stores domain events; ProgressService computes metrics; browser/CLI render results.

Methods: EventBus.subscribe(), publish(); ProgressProjector.update(); get_snapshot().

Collaboration: quiz/card/session events are saved independently of presentation. Snapshot reads accepted item scores and review ratings, computes means/sample sizes, and returns attempt/session/plan/event counts with an as-of timestamp.

Patterns: Observer, MVC. Changes: date ranges, trend charts, plan adherence and optional AI narrative are absent. Metrics are descriptive means, not calibrated probabilities of mastery. Video: F10 pending; contrast empty data and measured results.

F11 Study scheduling and reminders / UC11

Status: Replaced remote integration with local calendar export; deterministic.

Files/classes: core.py StudyAgentFacade validates reminders; CalendarAdapter formats RFC5545 ICS; SQLiteRepository persists stable session-based reminder IDs.

Methods: schedule(); CalendarAdapter.export(); SQLiteRepository.save().

Collaboration: approved plan and local time/timezone generate candidates. Rules check midnight, DST and overlaps across all local courses. Preview has no writes. Confirmation saves reminders idempotently; optional ICS export supplies events and a ten-minute alarm for manual calendar import. Failed export leaves local records.

Patterns: Command, Adapter, Facade, MVC. Changes: no live calendar authorization, remote conflict reads, remote retries, AI rescheduling or notification daemon. Video: F11 pending; show preview, confirmation, duplicate prevention, conflict and ICS.

F12 Orchestrated focused study session / UC12

Status: Modified; hybrid multi-step orchestration.

Files/classes: core.py AgentOrchestrator coordinates ToolRegistry, SessionState, ActiveState, PausedState, TerminalState, ProgressService and SQLiteRepository; learning.py provides grounded Q&A and quiz tools.

Methods: start(); run(); SessionState.handle(); ToolRegistry.execute(); LearningService.ask(), quiz(); get_snapshot().

Collaboration: course/goal and 5-120 minute budget create a saved learn checkpoint. Advance executes grounded explanation, then generates a quiz selected using measured weakness/difficulty, then saves review/progress. Each result is checkpointed. A failed tool pauses recoverably; pause/resume/skip/stop and expiry obey explicit legal states. The user can grade the generated quiz before advancing review.

Patterns: State, Facade, Command, Observer, Strategy, Adapter. Changes: bounded three stages replace arbitrary plans/Setup/activity mixes and continuous per-answer replanning. Paused time counts toward budget. Video: F12 pending; show actual tools, pause/restart, resume, review and terminal-state error.

3 Implemented design patterns

Pattern: MVC

Actual participants and roles: Browser view (`web/`), CLI boundary, `StudyController`, application/domain services, `SQLiteRepository` model storage

Problem solved and implementation evidence: Interfaces collect input and render output; controller validates course scope and dispatches use cases; shared services own rules. Changes to presentation do not duplicate grading or scheduling logic.

Stage 1 relationship: Retained at architectural level; `StudyDeskView` becomes a browser view, `StudyCLI` becomes module functions.

Pattern: Command

Actual participants and roles: `StudyCommand` is the command; facade/learning methods are receivers; `StudyController` is invoker; `CommandFactory` selects registered receivers

Problem solved and implementation evidence: `execute()` encapsulates request parameters independently of GUI/CLI. Allowlisting prevents arbitrary attribute invocation.

Stage 1 relationship: Generic receiver command replaces one class per use case. The factory itself is not counted as an extra pattern. Undo/history of commands is not implemented.

Pattern: Facade

Actual participants and roles: `StudyAgentFacade` is the shared application facade

Problem solved and implementation evidence: Coordinates learning, assessment, progress, plans, reminders, and sessions behind one application entry; interfaces do not call model or database directly.

Stage 1 relationship: Retained, with some orchestration consolidated into facade methods.

Pattern: Strategy

Actual participants and roles: `GenerationStrategy` template, `SummaryStrategy`, `GroundedAnswerStrategy`, `ConceptGraphStrategy`, `FlashcardStrategy`, `QuizStrategy`

Problem solved and implementation evidence: Each strategy supplies a task, output contract, and validator while sharing evidence checks and one bounded validation retry. `LearningService._generate()` selects it.

Stage 1 relationship: Retained for generation. Separate adaptive strategy objects were not added; adaptation policy is in `StudyAgentFacade.adapt()`.

Pattern: Adapter

Actual participants and roles: `ModelGateway` protocol and `OpenAIModelAdapter`; `CalendarAdapter` converts reminders into iCalendar

Problem solved and implementation evidence: Provider HTTP/output structure and calendar format remain outside feature policies. Tests can inject a fake model or fake HTTP opener.

Stage 1 relationship: OpenAI adapter retained. Remote calendar adapter replaced by local ICS exporter.

Pattern: Observer

Actual participants and roles: `EventBus` subject; `ProgressProjector.update()` observer; learning/assessment/session methods publishers

Problem solved and implementation evidence: Save activity events without coupling their producers to dashboard rendering; `ProgressService.get_snapshot()` deterministically projects accepted observations.

Stage 1 relationship: Retained, but snapshots are computed on request from saved attempts/events rather than continuously materialized into a separate progress repository.

Pattern: State

Actual participants and roles: `SessionState` abstraction; `ActiveState`, `PausedState`, `TerminalState` concrete states; `AgentOrchestrator.run()` context

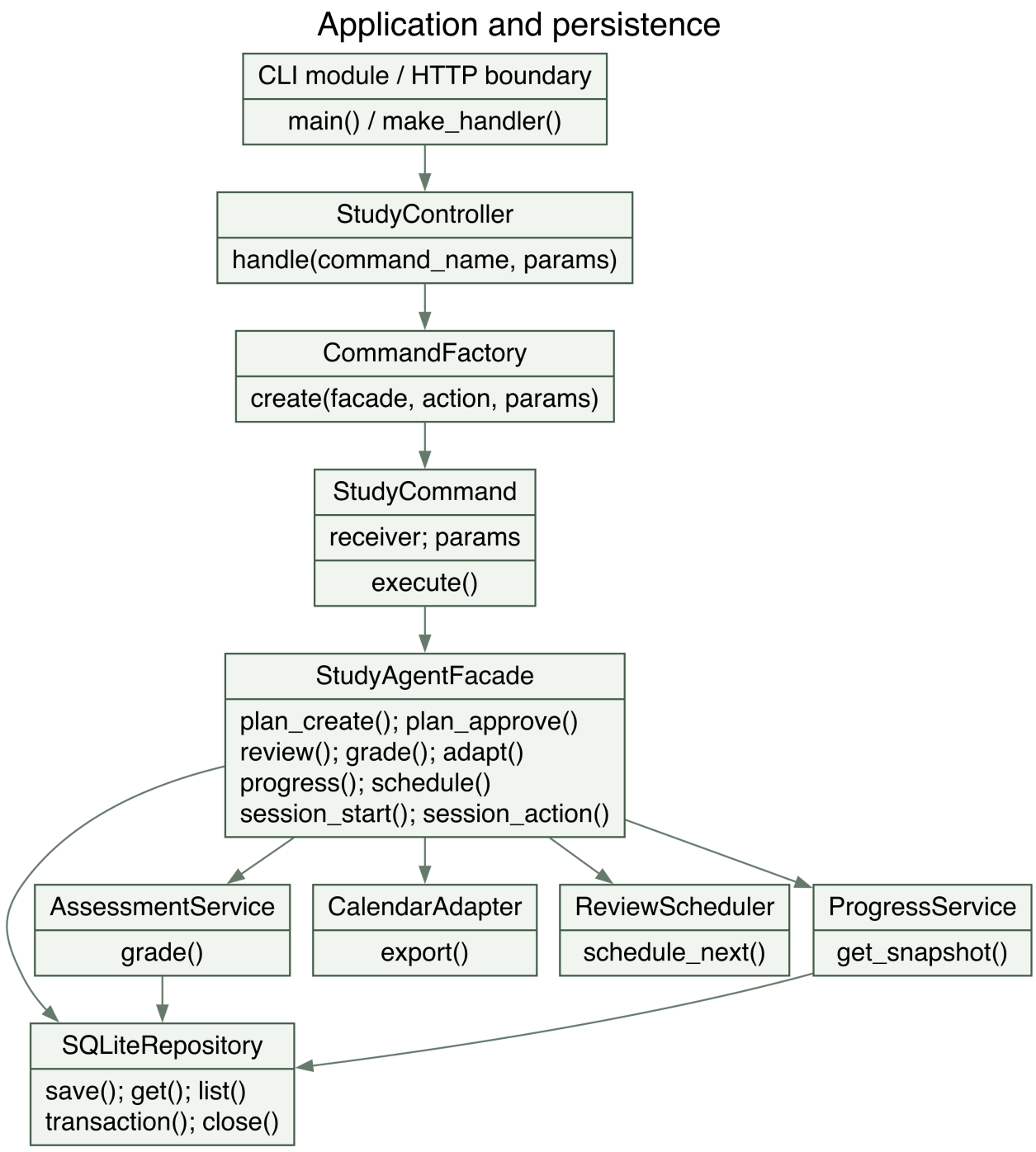
Problem solved and implementation evidence: Legal pause/resume/advance/skip/stop transitions are delegated to state objects. Failure persists a recoverable pause; terminal states reject further actions.

Stage 1 relationship: Retained with shared active behavior for learn/practice/review; no separate class for every activity state.

Updated UML class diagrams

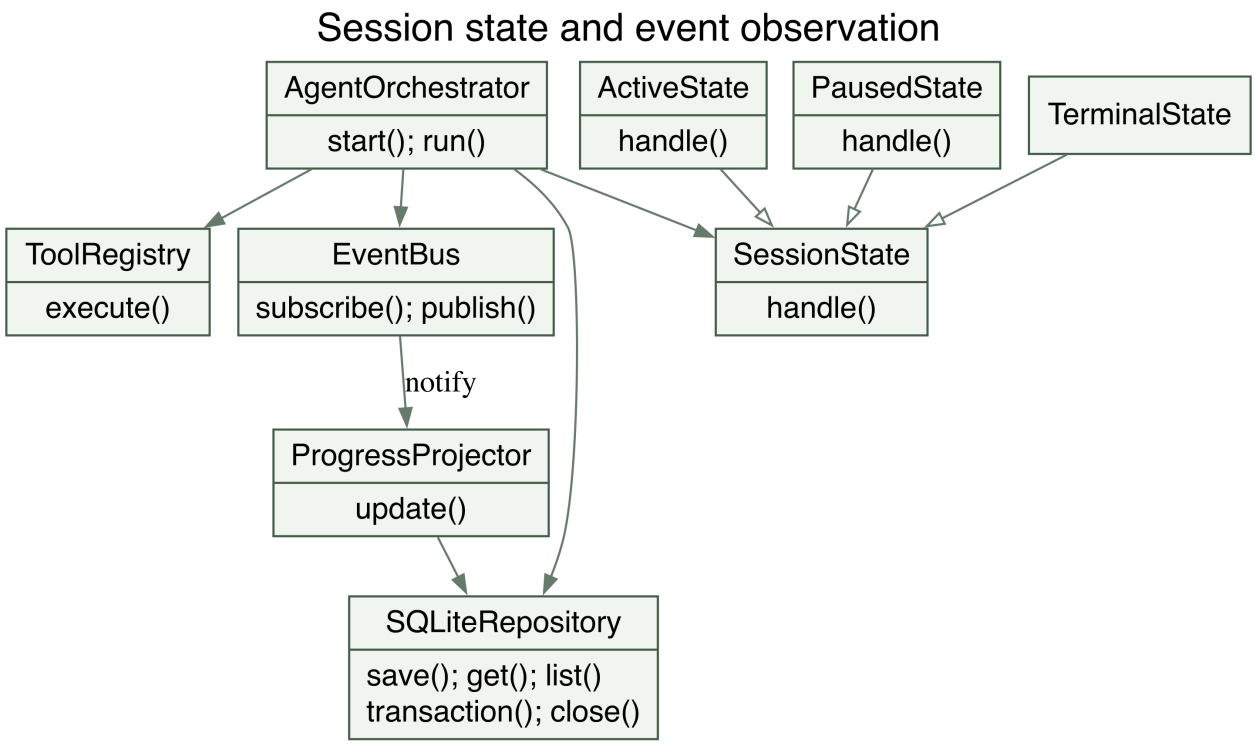
Figures split the same implementation into four readable views. Solid arrows show dependencies; open triangle arrowheads show inheritance; dashed open triangles show protocol implementation. Repeated classes are references to the same class. CLI/HTTP are module boundaries, not invented Python classes. The complete editable relationship model and updated use-case diagram are in docs/architecture.md.

Application boundary, commands, facade and persistence



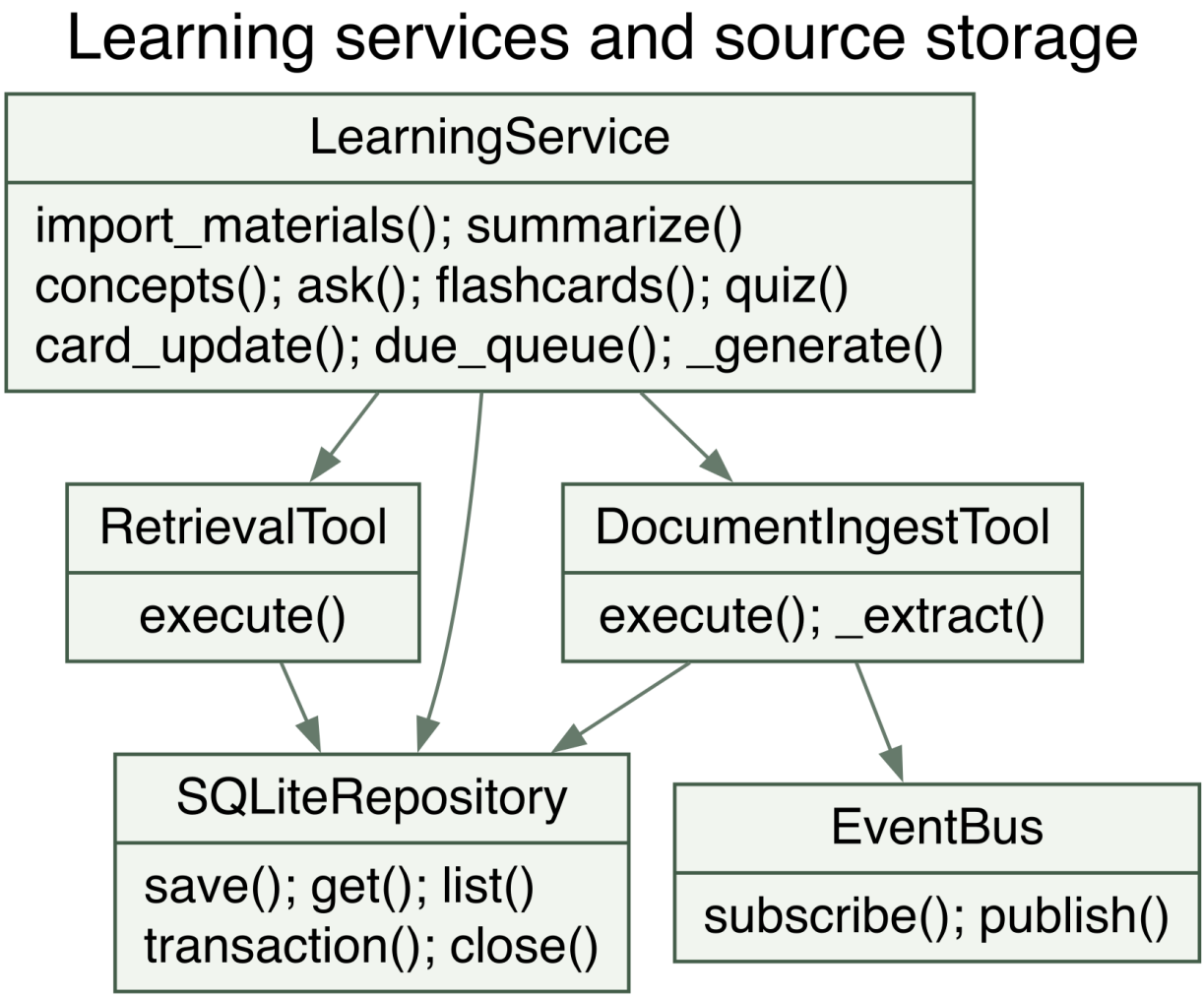
Editable source: docs/diagrams/application.dot. The full source-level relationship diagram is also available in docs/architecture.md.

Session State participants, tools and Observer participants



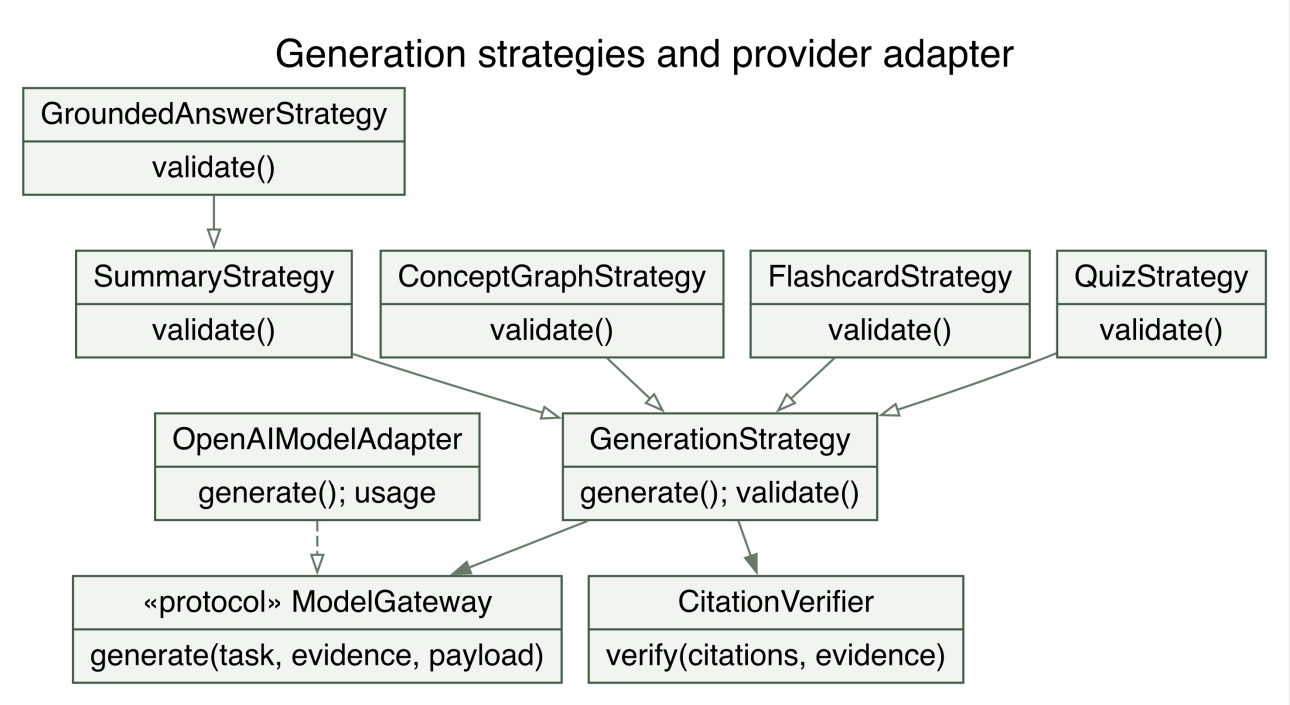
Editable source: docs/diagrams/runtime.dot. The full source-level relationship diagram is also available in docs/architecture.md.

Learning services, source ingestion and scoped retrieval



Editable source: docs/diagrams/learning.dot. The full source-level relationship diagram is also available in docs/architecture.md.

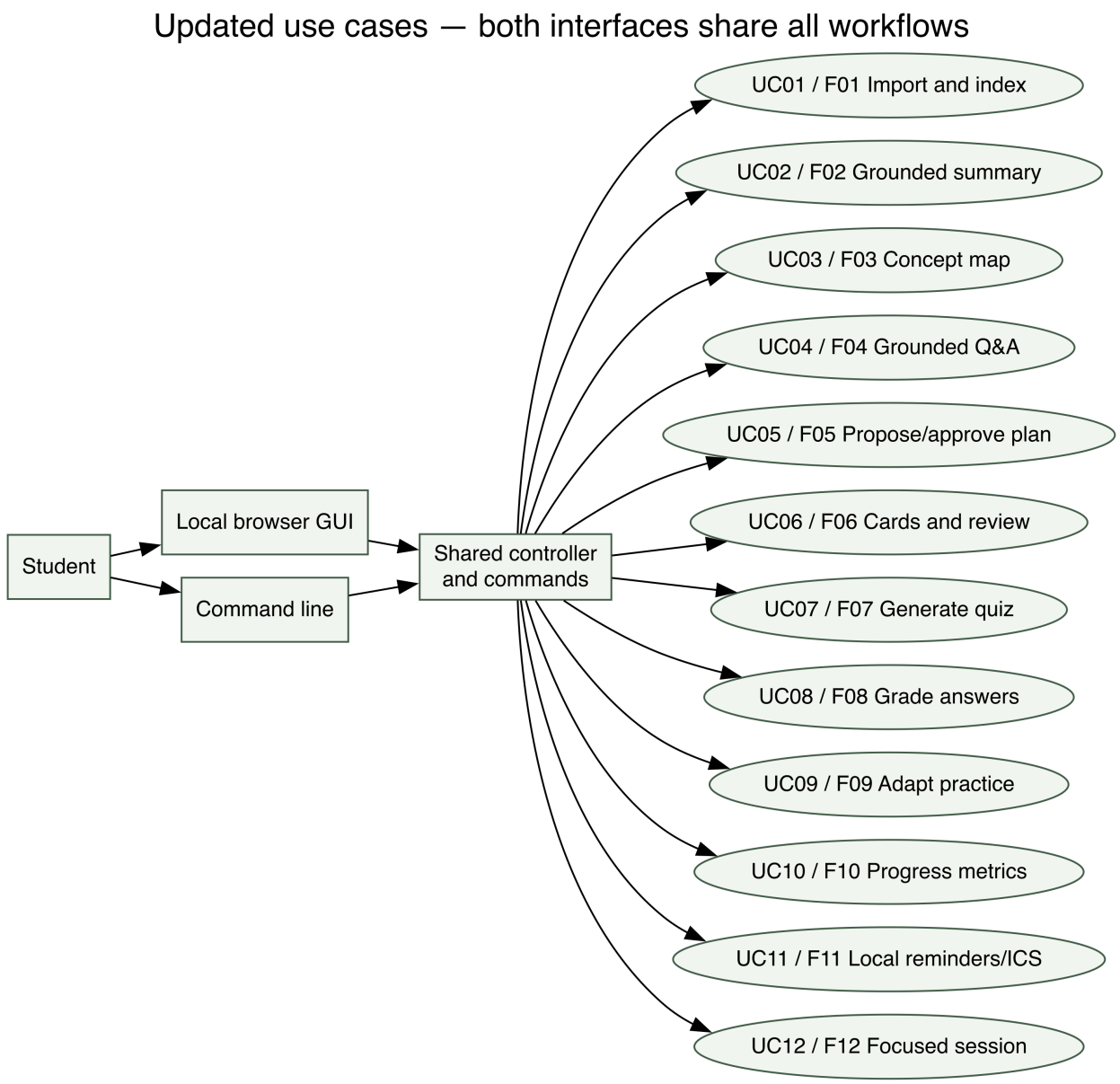
Generation Strategy participants and model Adapter



Editable source: docs/diagrams/strategies.dot. The full source-level relationship diagram is also available in docs/architecture.md.

Updated use-case diagram

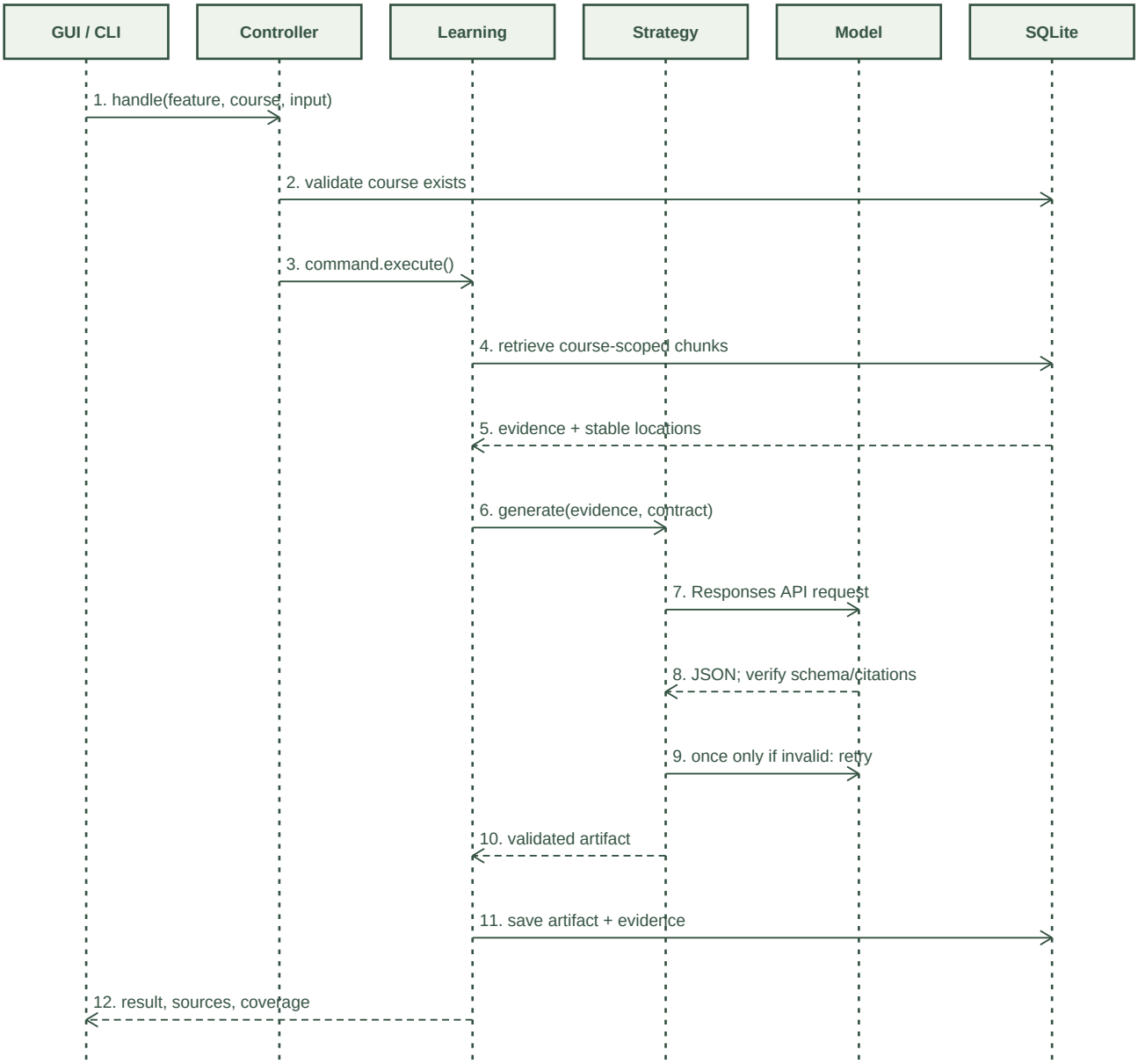
The same student can invoke UC01-UC12 through either local interface. Source ingestion uses local files; generated learning activities use the OpenAI adapter; reminders are local ICS exports for manual import, replacing the remote-calendar actor from Stage 1.



Updated grounded-generation sequence

Applies to F02/F03/F04/F06/F07. A retrieval gap stops before the model call; invalid output is retried once and never persisted if still invalid.

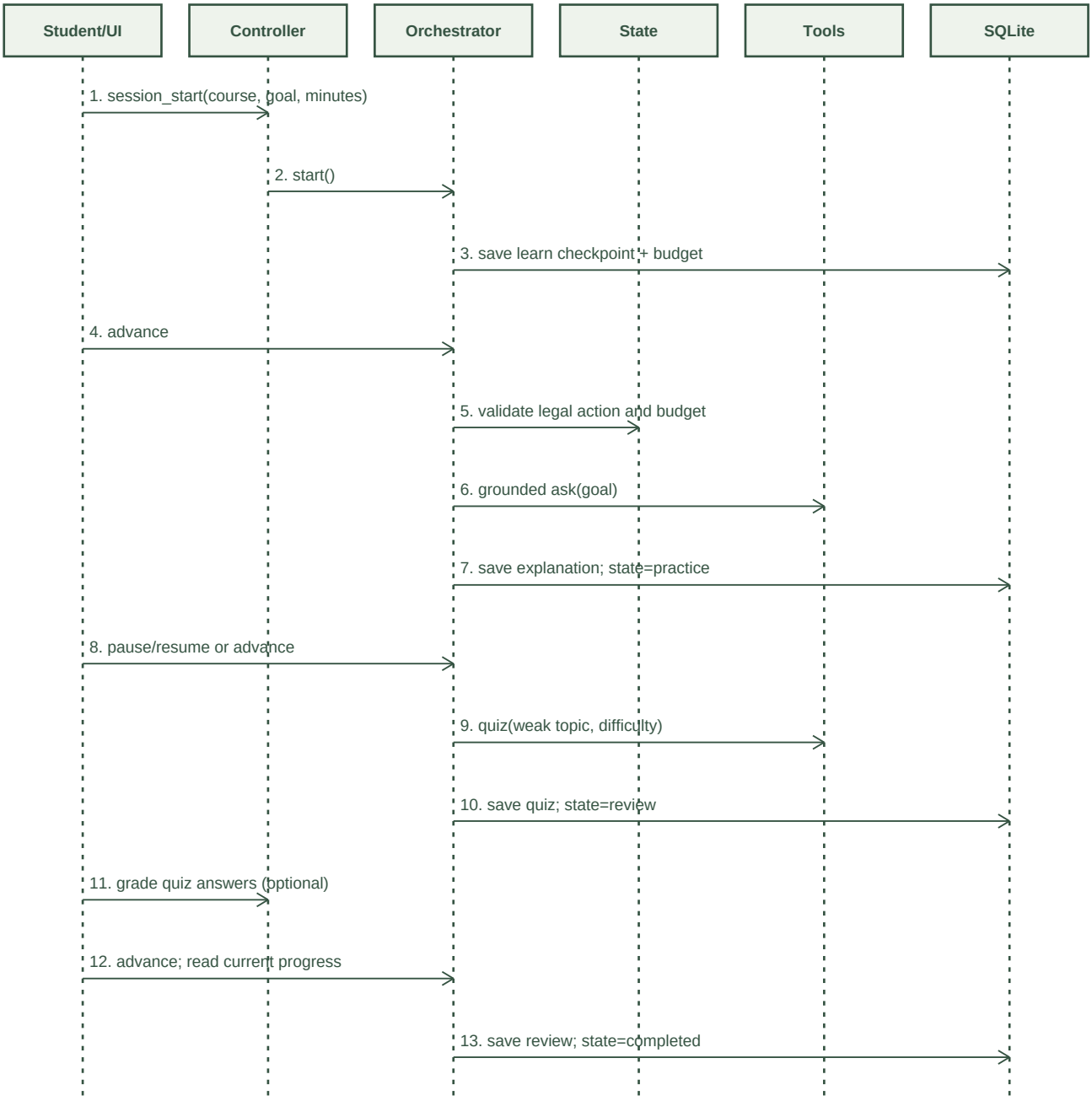
Grounded generation / principal success path



Updated focused-session sequence

F12 now runs three bounded stages. Pause/resume/skip/stop are legal at checkpoints. Tool failures persist a paused state; expiry ends work before another tool is called. The student submits generated quiz answers through ordinary grading before review when desired.

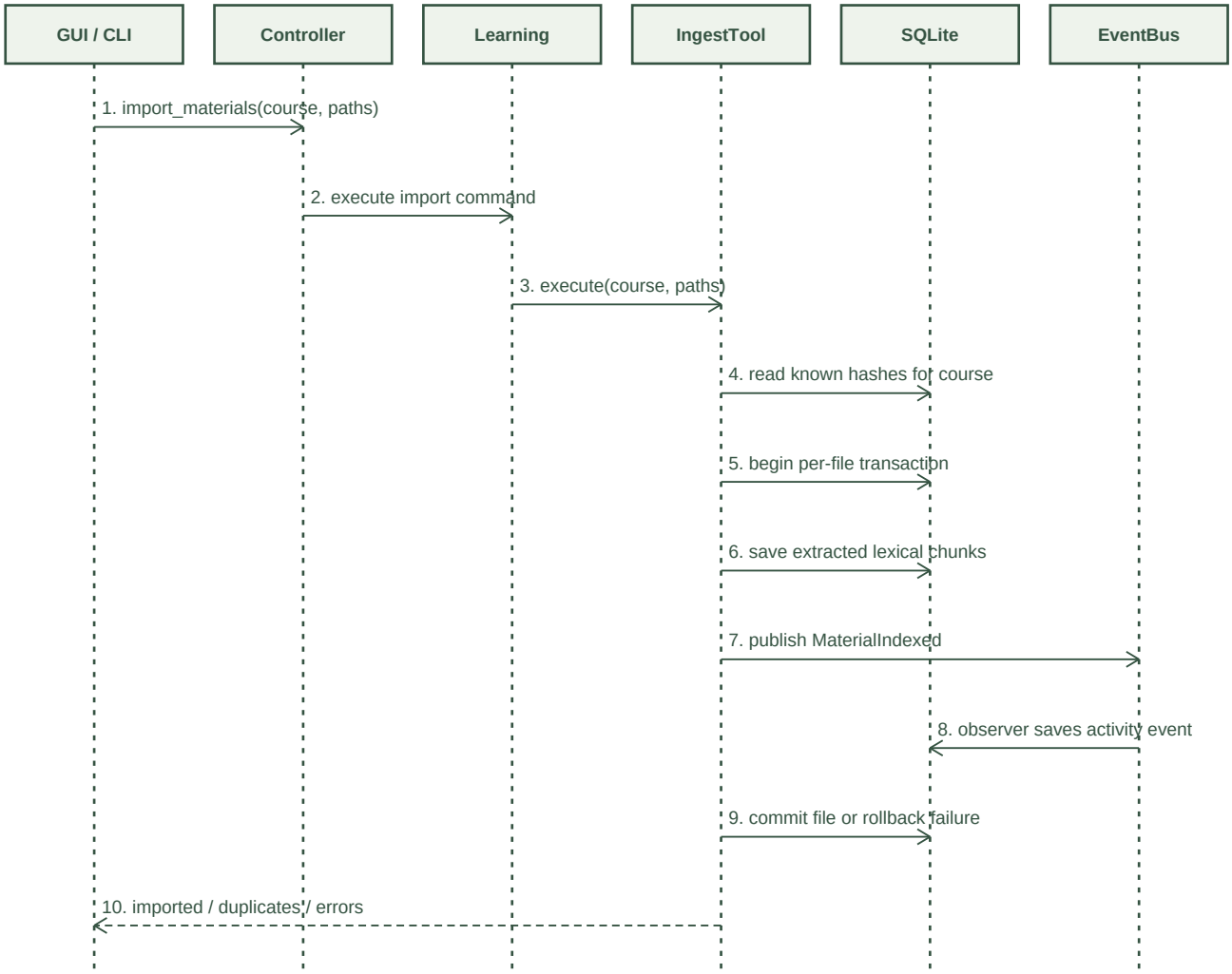
Focused session / checkpointed activity path



Updated sequence: F01 / SD01 Ingest and index

Each file has its own transaction. Duplicate hashes are reported without new records. Unsupported, encrypted, empty or oversized files produce per-file errors; a parser/event failure rolls back that file. No embeddings or model-generated metadata are claimed.

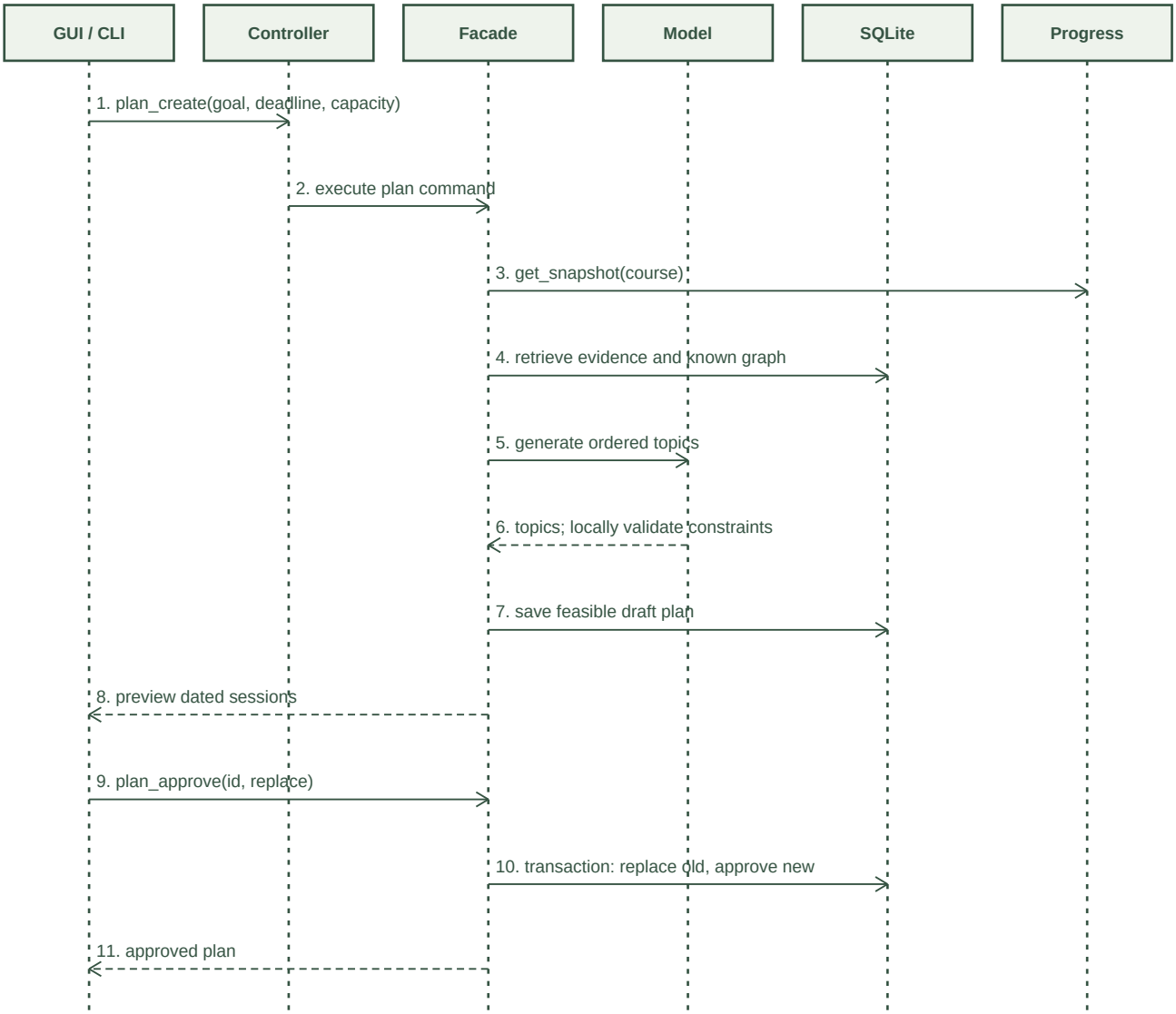
F01 / SD01 Ingest and index



Updated sequence: F05 / SD03 Plan, validate and approve

The model orders evidence-linked topics using goal and mastery. Deadline, daily capacity and known graph prerequisites are enforced locally. Infeasible plans fail before saving; replacing an approved plan requires explicit replacement confirmation.

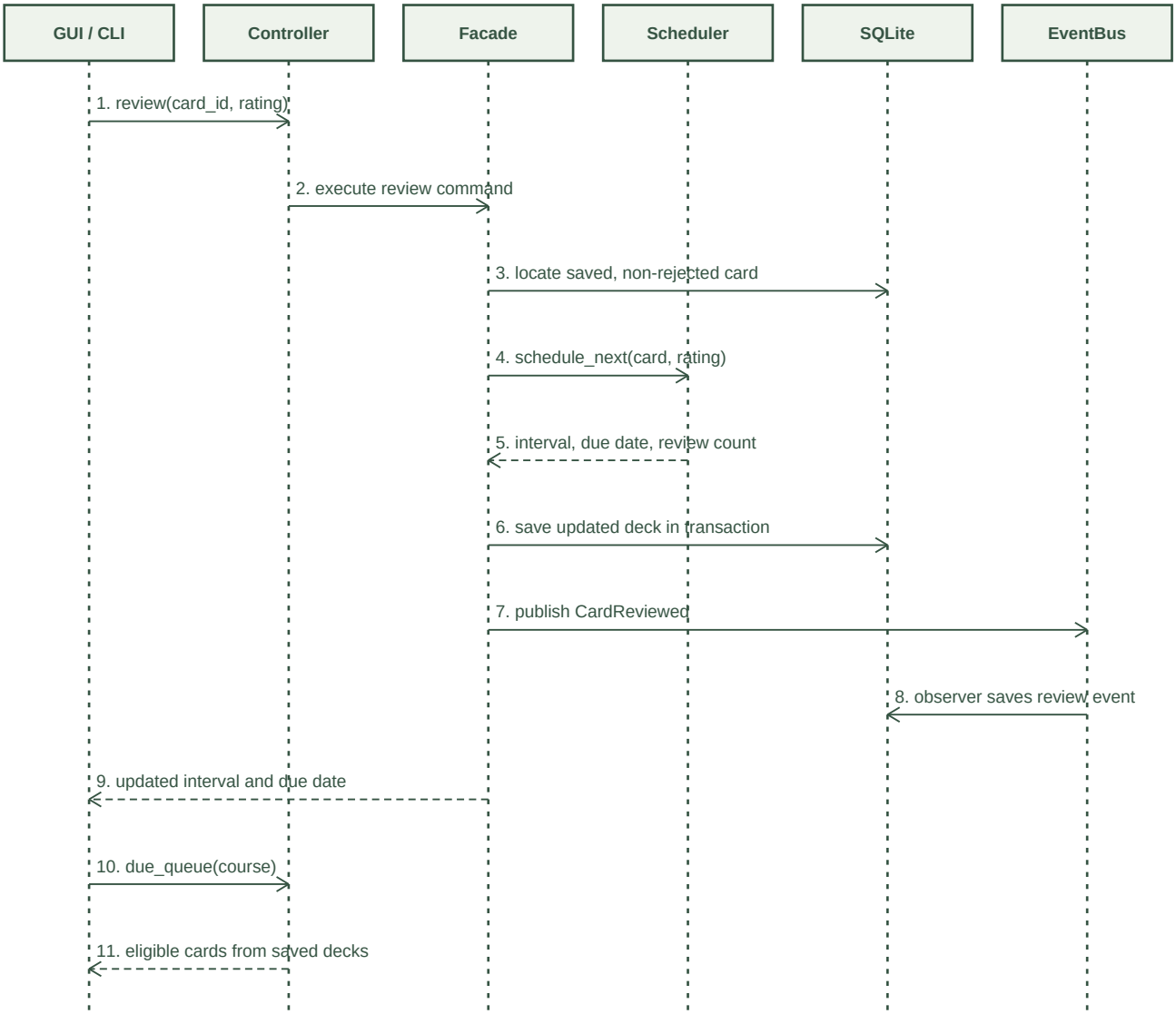
F05 / SD03 Plan, validate and approve



Updated sequence: F06 / SD11 Review and next due date

Only ratings 0-3 are accepted. Rejected cards cannot be reviewed. The updated card and CardReviewed event are saved atomically. Due queues exclude rejected cards and cards scheduled after the requested date.

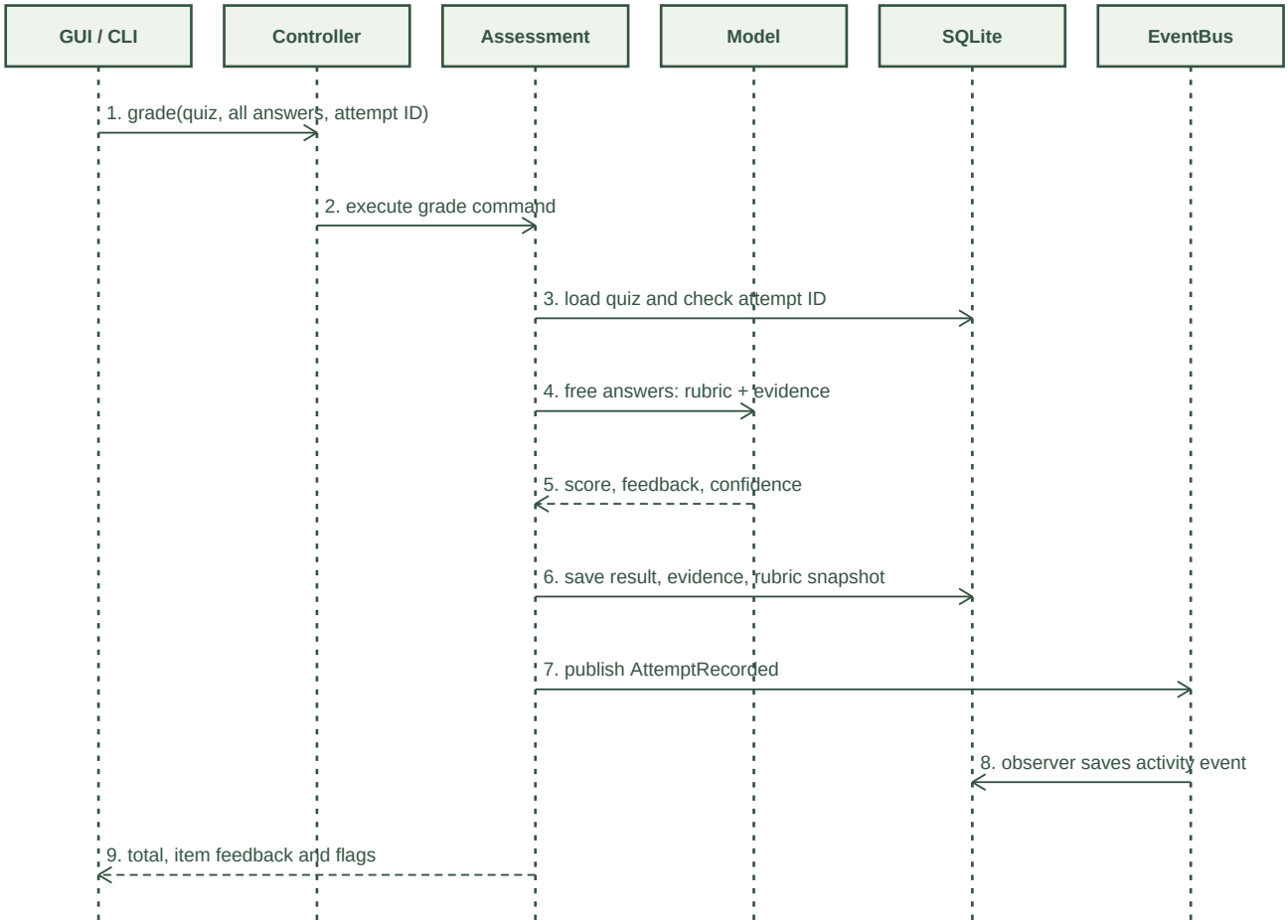
F06 / SD11 Review and next due date



Updated sequence: F08 / SD05 Grade and retain rubric

Completeness and attempt-ID duplicate checks happen before scoring. Choice grading is deterministic. Free-text grades must have bounded score/confidence and nonempty feedback. Low-confidence items are flagged and excluded from mastery; invalid results roll back the attempt.

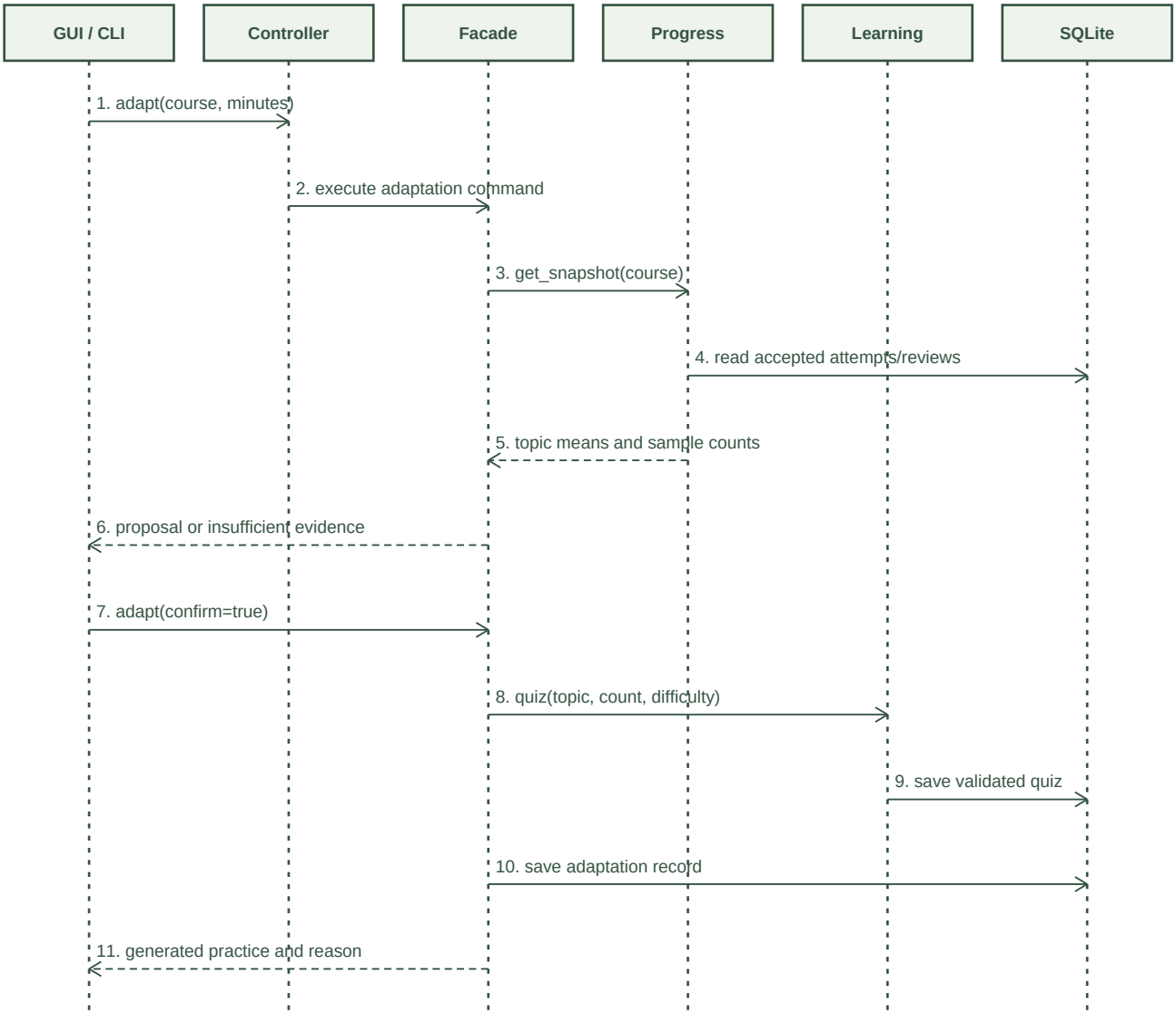
F08 / SD05 Grade and retain rubric



Updated sequence: F09 / SD09 Diagnose and confirm practice

At least two observations on a topic and mastery below 0.7 are needed. Otherwise no diagnosis or generation is claimed. Confirmation generates a quiz using the chosen difficulty; adaptation does not silently replace an active plan.

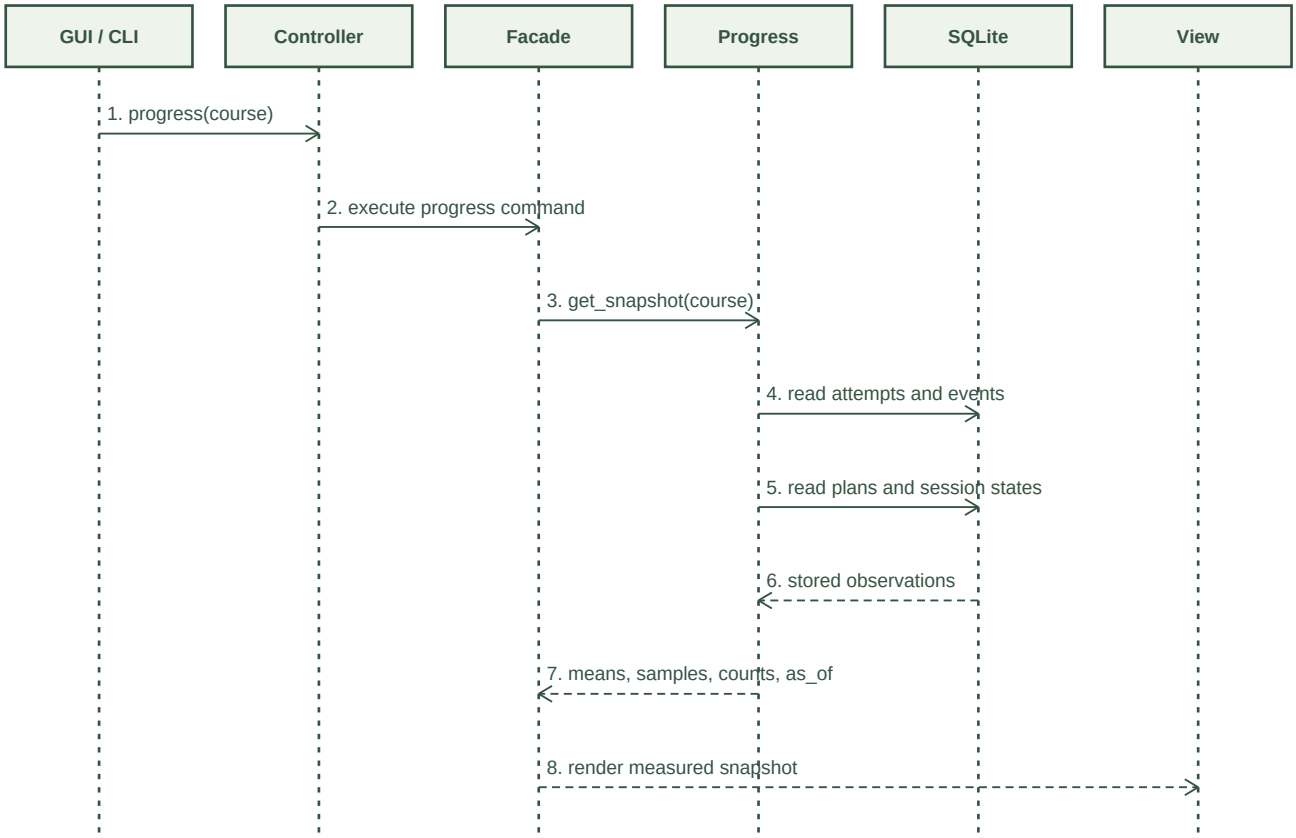
F09 / SD09 Diagnose and confirm practice



Updated sequence: F10 / SD10 Compute progress snapshot

The snapshot derives deterministic means and counts from saved attempts and events. Low-confidence grade items are excluded. Empty data returns empty topics and zero counts; no trend or calibrated mastery claim is invented.

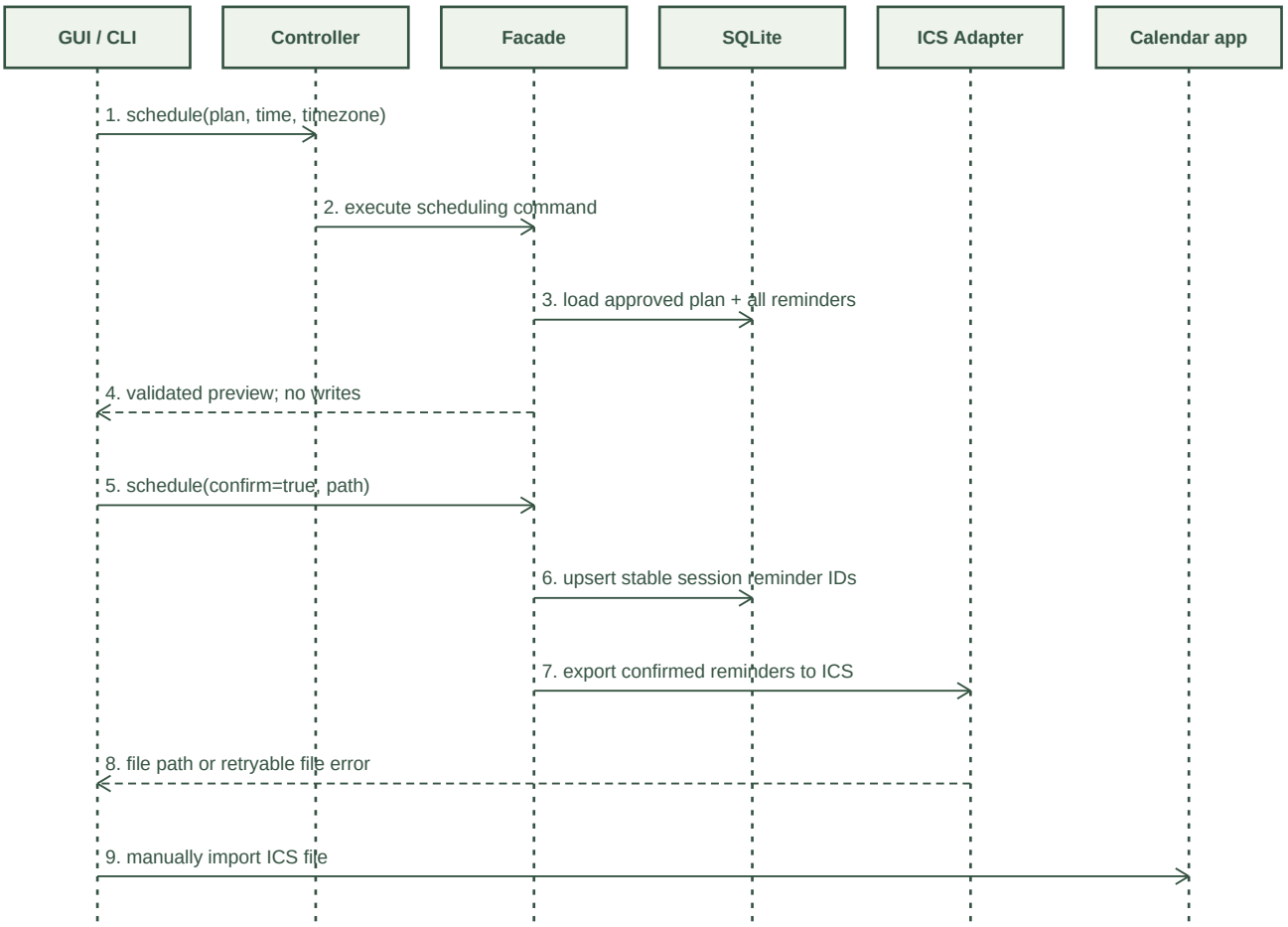
F10 / SD10 Compute progress snapshot



Updated sequence: F11 / SD08 Confirm local reminders and export

This replaces the original remote-calendar sequence. Only approved plans are scheduled. Candidate dates are checked for timezone/DST, midnight and conflicts across all local courses. Preview does not write; confirmation uses stable reminder IDs. An export failure leaves confirmed local reminders available for retry. Manual calendar import is a user step.

F11 / SD08 Confirm local reminders and export



4 Changes and rationale

Original design: F01 hybrid AI titles/tags, local vector embeddings; `DocumentIngestTool`, named material repository

Actual implementation: TXT/MD/PDF/DOCX parsing, SHA256 duplicates, overlapping chunks, filename titles/tags, lexical retrieval; `DocumentIngestTool`, `RetrievalTool`, generic repository

Reason: Avoid an embedding service dependency and make import deterministic and locally inspectable

Impact and remaining limits: AI metadata inference and vector embeddings are not implemented. Scanned PDFs need OCR outside the application. PDF/DOCX formatting is reduced to text. Per-file save plus event is transactional.

Original design: F02 selected-section summaries, concise/detailed content, evidence checking

Actual implementation: `SummaryStrategy`, material selection, detail and optional focus at service level; persisted retrieved evidence and partial-coverage metadata

Reason: Bounded prompts and common validation pipeline

Impact and remaining limits: No section-selector domain object or exhaustive whole-course summarization; GUI/CLI option exposure may be narrower than service arguments. Citations validate references, not semantic truth.

Original design: F03 confidence-bearing knowledge map, quarantine unsupported edges

Actual implementation: `ConceptGraphStrategy` normalizes duplicate names, checks confidence/endpoints/relation types, rejects invalid generated graph and retries

Reason: Keep the graph internally consistent and preserve evidence

Impact and remaining limits: Invalid relations are rejected rather than placed in a quarantine workflow. Regeneration saves another graph record; there is no graph-version comparison service. Browser renders graph nodes/edges and evidence data; no dedicated graph-editing domain service.

Original design: F04 `IntentRouter`, `TaskPlanner`, grounded strategy and conversation repository

Actual implementation: Explicit feature command, ranked lexical retrieval, `GroundedAnswerStrategy`, last four stored turns, shared SQLite store

Reason: Predictable, testable course-scoped execution without a second model routing request

Impact and remaining limits: No model intent classifier, query decomposition, or autonomous retrieval-plan search. Empty retrieval is an explicit gap; ambiguous contextual questions may fail lexical retrieval.

Original design: F05 learner profile, priorities, availability windows, prerequisite plan validator, approval

Actual implementation: `StudyAgentFacade.plan_create()` obtains ordered topics from model using goal retrieval or course-wide fallback for broad goals, validates daily-minute capacity/deadline/known graph ordering, builds dated slots; `plan_approve()` requires replacement confirmation

Reason: Feasible minimum local planner with explicit approval and deterministic safeguards

Impact and remaining limits: No learner-profile class, arbitrary availability windows, priority weights, detailed activity/checkpoint generation, or automatic alternative proposal. Known graph constraints apply when matching topics exist.

Original design: F06 generated cards, duplicate filters, review scheduler, edit/reject

Actual implementation: `FlashcardStrategy`, `ReviewScheduler`, `LearningService.card_update()` and `due_queue()`, `review()` with 0-3 ratings

Reason: Retain source-linked practice and a deterministic, understandable interval rule

Impact and remaining limits: Scheduler is a simple rating-based interval policy, not a calibrated learning algorithm. User edits are marked `user_edited`; their semantics are not reverified by an LLM. Rejected cards cannot be reviewed.

Original design: F07 objective/free-response quiz, objectives/format/difficulty/coverage validation, partial quiz fallback

Actual implementation: `QuizStrategy` validates requested count, uniqueness, choice options/key, free rubric, topic, difficulty and citations

Reason: Prevent malformed quizzes and persist full evidence

Impact and remaining limits: Explicit objective-coverage validation, user-selected format mix, partial-quiz fallback and a separate `QuizValidator` are absent. Invalid count or repeated invalid output fails safely.

Original design: F08 attempts, deterministic objective grading, rubric LLM grading, confidence review

Actual implementation: `AssessmentService.grade()` validates complete answers, exact normalized objective key, free-response score/feedback/confidence, stores rubric snapshot, excludes low-confidence grades from mastery

Reason: Preserve auditable scoring with a deterministic objective path

Impact and remaining limits: Low-confidence results are flagged, but no human-grade override workflow is implemented. Duplicate protection depends on reusing attempt ID; a new ID is a separate attempt. There is no independent `Attempt` domain class.

Original design: F09 prerequisite-aware strategy selection, AI explanation, confirmation before plan insertion

Actual implementation: `StudyAgentFacade.adapt()` requires two observations, identifies lowest weak topic, sets difficulty, confirms before generating quiz

Reason: Avoid confident diagnosis from a single result and reuse verified quiz generation

Impact and remaining limits: Recommendation explanation is deterministic, not AI narrative. No prerequisite-remediation traversal, separate `AdaptationService`/strategy, or active-plan insertion. Confirmed practice is saved as quiz plus adaptation record.

Original design: F10 analytics projections, mastery, adherence, date ranges, trends and optional narrative

Actual implementation: `ProgressProjector` stores activity events; `ProgressService` computes accepted item/card means, sample sizes, attempt count, completed sessions and event count

Reason: Reproducible small local analytics without invented measurements

Impact and remaining limits: Plan adherence, date filtering, trend series/charts and AI narrative are not implemented. Metrics are descriptive sample averages, not validated mastery estimates.

Original design: F11 remote calendar authorization/writes, rescheduling, retryable pending records

Actual implementation: `schedule()` previews/approves reminders, validates timezone/overlaps across courses, uses stable session-based IDs, optionally calls `CalendarAdapter.export()`

Reason: Deliver useful scheduling without requesting third-party credentials or external writes

Impact and remaining limits: Local ICS replacement; manual import required. No live calendar conflict read, background notifications, AI rescheduling, remote retries or authorization. Export filesystem failure leaves confirmed local reminders available for retry.

Original design: F12 arbitrary typed plan, Setup/Learn/Practice/Review, retries and adaptation after activities

Actual implementation: `AgentOrchestrator`, `ToolRegistry`, three-stage workflow, actual learning tool calls, persisted checkpoints, `SessionState` objects, legal controls and wall-clock budget

Reason: Bound scope and make recovery observable and testable

Impact and remaining limits: No separate Setup state, arbitrary activity mix, per-answer continuous replanning or autonomous scheduling. Paused time counts toward wall-clock budget. See session sequence above.

Original design: Separate classes/repositories for Course, Material, Plan, Quiz, Attempt, Session and snapshots

Actual implementation: JSON aggregate records and one `SQLiteRepository` with kind/course indexing and transactions

Reason: Reduce unused indirection and keep storage behavior consistent

Impact and remaining limits: Aggregate invariants live in service validators rather than domain model constructors. Report and diagrams reflect actual classes.

Original design: Many concrete `ImportMaterialCommand`, `GradeAttemptCommand`, etc.

Actual implementation: One receiver-based `StudyCommand`, registered `CommandFactory`

Reason: Every use case already has a cohesive method; avoids empty wrappers

Impact and remaining limits: Command names remain explicit; no named per-feature command classes or undo implementation.

Original design: `IntentRouter`, `TaskPlanner`, `PlanValidator`, `ConceptGraphService`, `AdaptationService`, named strategy for every operation

Actual implementation: Command allowlist; facade policies; generation strategy validators; `AgentOrchestrator`

Reason: Consolidate rules around actual operations rather than create unused diagram classes

Impact and remaining limits: The original class names and some broader behavioral promises are intentionally not claimed.

Original design: Performance cache keyed by provider/schema/content versions

Actual implementation: No generation cache

Reason: Avoid stale artifact reuse before cache invalidation requirements are settled

Impact and remaining limits: Repeated generation incurs model calls and cost; only saved artifacts are reusable manually.

5 AI tools and collaboration log

AI collaboration, verification, and weekly development record

Tool use and accountability

Tool: Codex desktop coding agent

Model: GPT-6 family as described by this environment; exact billed model identifier was not exposed

Work performed: Read Stage 1 and Stage 2 documents; inspect/fetch original repository; implement shared core, model adapter, GUI/CLI, tests, sample configuration, documentation and report preparation; coordinate implementation and cross-review

Token and money cost: Unavailable in this environment. Do not interpret unavailable as zero. The account usage/billing export is needed for a final measured total.

Tool: Codex delegated coding agents

Model: Same inherited model as parent; exact billed identifier unavailable

Work performed: Separate learning/retrieval, application core, and interface tasks; shared source review and regression fixes

Token and money cost: Included in unavailable agent-session accounting; no invented per-agent estimate.

Tool: StudyForge runtime OpenAI adapter

Model: Default configured model `gpt-4.1-mini-2025-04-14`, user-overridable through `OPENAI_MODEL`

Work performed: Production implementation for summaries, answers, concepts, cards, quizzes, study-plan topics and free-response grading

Token and money cost: No real API key was supplied during the initial implementation checks. Live feature execution and actual runtime token/cost measurements remain unverified. In-memory `usage` counts API-reported input/output tokens when a real response occurs; money is not calculated or stored.

Local Python tools, SQLite, Git, PDF extraction, and browser/HTTP checks are development tools, not additional LLM models. Unit-test fake gateways produce deterministic test fixtures only. They are dependency-injected inside tests and are not a production fallback or a claim that an LLM ran.

The student remains responsible for understanding, reviewing, integrating, validating, recording demonstrations, and submitting the result. AI-produced code does not transfer that responsibility.

Representative collaboration record

The following are concise summaries of representative requests and outcomes from this implementation session, not fabricated full conversation transcripts.

Interaction: Read all Stage 2 requirements, retrieve Stage 1 repository, and account for all original features

Development result and review outcome: Read the five-page Stage 2 PDF and the forty-page Stage 1 PDF; retained F01-F12 and recorded deviations instead of silently omitting proposed behavior.

Interaction: Implement learning ingestion/retrieval and evidence-grounded strategies with strict contracts

Development result and review outcome: Added PDF/DOCX/TXT/MD extraction, duplicate detection, scoped lexical retrieval, stable source chunks, summaries, concept graphs, Q&A memory, cards and quizzes. Strategy-specific validators reject bad structure/citations and retry once.

Interaction: Implement one shared controller, local persistence, model adapter, plans, review, grading, analytics, scheduling and sessions

Development result and review outcome: Added SQLite transactions, receiver commands, facade services, observer events, legal session states and real OpenAI Responses HTTP integration. Model failures surface safe errors rather than canned study answers.

Interaction: Build GUI and CLI over the same requests

Development result and review outcome: Added local browser workspace and CLI command parser; HTTP Host/Origin/token checks restrict local mutation requests. Single-thread HTTP handling avoids interleaving one SQLite connection's transactions.

Interaction: Cross-review boundaries and core invariants

Development result and review outcome: Found that `handle(action, **params)` collided with session parameter `action`; renamed first argument to `command_name` and added regression.

Interaction: Cross-review adaptive practice and reminders

Development result and review outcome: Found difficulty was initially metadata only; passed actual difficulty into quiz generation. Expanded calendar conflicts from one course to all local course reminders.

Interaction: Cross-review import transaction behavior

Development result and review outcome: Found that an event-observer failure after material save could report failure while leaving the material saved. Added per-file repository transaction and rollback regression.

Interaction: Review model failure retries

Development result and review outcome: Distinguished malformed model JSON (`InvalidModelOutput`) from missing-key/network/HTTP errors so the bounded schema retry does not repeat unrelated failed requests.

Interaction: Challenge state-machine edge cases

Development result and review outcome: Found expiry could bypass legal-state validation; validate state first and regression-test that an expired completed session cannot advance again. Tool failure persists pause and resume state.

Interaction: Review planning and progress against original report

Development result and review outcome: Added known graph prerequisite-order checks, rejected duplicate topics, added broad-goal planning fallback with regression, and documented omitted availability windows, trend analytics, adherence, remote calendar integration and general-purpose autonomous planning.

Verification evidence and limits

The repository's test suite is the executable record. Run the current suite using the README command; the exact current count should come from that run rather than a stale count in this log. Coverage includes repository transactions/course isolation, parser behavior, import failures and duplicate hashes, evidence scoping/citation validity, generation schema retry, graph relationships, card review/edit/reject, quiz schema/grading, confidence exclusion, plan feasibility/approval/prerequisites, reminder conflicts/idempotency/ICS escaping, session transitions/recovery/expiry, actual controller arguments, CLI request mapping, local HTTP contracts and adapter HTTP construction.

A fake HTTP opener verifies the production adapter sends the expected Responses request, parses provider-shaped output, counts reported tokens and redacts secret-bearing error details. This is an adapter contract test, not a live model test. Test fakes never constitute demonstration evidence for AI feature quality.

The original main branch contained documentation only, so there was no previous runnable application whose behavior could be compared. The relevant baseline comparison is Stage 1 feature/class/interaction promises against the new source. `architecture.md` records those differences explicitly. New regression tests demonstrate corrected behavior for defects found during implementation; there is no claim of Stage 3 KUMA validation.

No API key was provided to the implementation agents. Live LLM feature acceptance and output-quality evaluation still require local credentials, configured model access and a successful real execution. Feature demonstration recordings and accessible timestamped video links are separate requirements; a test run, screenshot, code listing or generated report is not a replacement for those recordings.

6 Development and verification summary

Weekly code activity, Toronto local dates

The fetched original public repository history contained exactly two commits, both on October 9, 2026 Toronto local time:

Commit: `b4ab728`

Toronto date: October 9, 2026

Description: `docs(stage1): publish project design report`

Qualifies for Stage 2 weekly code minimum?: No: documentation-only and outside all reporting periods.

Commit: `ee6d35a`

Toronto date: October 9, 2026

Description: `docs(stage1): align design traceability`

Qualifies for Stage 2 weekly code minimum?: No: documentation-only and outside all reporting periods.

Required reporting period, inclusive: October 20-26, 2026

Required meaningful code commits: 5

Qualifying commits found in original repository history: 0

Status: Not met

Required reporting period, inclusive: October 27-November 2, 2026

Required meaningful code commits: 5

Qualifying commits found in original repository history: 0

Status: Not met

Required reporting period, inclusive: November 3-9, 2026

Required meaningful code commits: 5

Qualifying commits found in original repository history: 0

Status: Not met

Implementation performed in this session is current work after those periods. New implementation commits retain their actual creation dates. The required historical development and contemporaneous pushes are not established by the available history. Existing history was preserved.

The Stage 2 PDF states that failure of one or more periods incurs one total five-point deduction, not five points per period. This log reports the gap; it does not claim a waiver, instructor approval, or retrospective compliance. The due date stated in the assignment was November 10, 2026. Repository preparation after that date does not establish an on-time submission. Final repository publication, report status, video links and course-portal submission must be checked independently.

Current implementation commits

These are actual implementation commits available when this report was generated, not substitutes for missing historical activity. The report-packaging commit itself is not listed. 5cec58e / 2026-11-12T19:37:30-05:00 / build(runtime): declare portable timezone and report dependencies

7351349 / 2026-11-12T19:27:47-05:00 / feat(interfaces): expose study workflows through GUI and CLI

5cb98a0 / 2026-11-12T19:27:44-05:00 / feat(learning): add source-grounded ingestion and practice strategies

f1f18ba / 2026-11-12T19:27:42-05:00 / feat(agent): implement persistent study workflows and model adapter

c0dbde0 / 2026-11-12T19:27:39-05:00 / build(project): add reproducible Python setup and verification

Reproduction and remaining gates

Run python -m unittest discover -s tests -v and make check after installing with python -m pip install -e . . The suite passed 39 tests at the final verification recorded for this report. Python compilation, JavaScript syntax, credential scanning, package installation and PDF visual checks are separate release checks. No Stage 3 KUMA validation is claimed.

A production launch without credentials must show a configuration error; test fixtures are never accepted as live LLM evidence. Record actual live results before claiming final feature acceptance. Confirm all source/report files are pushed to the public repository and verify video access signed out before course submission.

7 Demonstration index

The owner selected the recording-guide workflow. No videos have yet been recorded or supplied in this session. All twelve rows remain pending; no accessible link or real timestamp is invented. docs/demonstrations.md contains exact GUI/CLI invocation, input, expected result, alternative/error and AI explanation for each segment. docs/demo-index.json is the machine-readable index to update after uploading.

A combined recording is acceptable only when each feature has a direct timestamped link. Record the running application with the real provider and show both GUI and CLI collectively; screenshots, this report and test fixtures are not video demonstrations. Avoid recording .env or account credentials.

F01 Material ingestion and indexing / UC01

Status: awaiting owner recording and upload. Video link: not supplied. Actual timestamp: not supplied. Planned segment: F01. See the same feature heading in docs/demonstrations.md.

F02 Grounded summaries / UC02

Status: awaiting owner recording and upload. Video link: not supplied. Actual timestamp: not supplied. Planned segment: F02. See the same feature heading in docs/demonstrations.md.

F03 Concept extraction and knowledge map / UC03

Status: awaiting owner recording and upload. Video link: not supplied. Actual timestamp: not supplied. Planned segment: F03. See the same feature heading in docs/demonstrations.md.

F04 Grounded question answering / UC04

Status: awaiting owner recording and upload. Video link: not supplied. Actual timestamp: not supplied. Planned segment: F04. See the same feature heading in docs/demonstrations.md.

F05 Personalized study plan / UC05

Status: awaiting owner recording and upload. Video link: not supplied. Actual timestamp: not supplied. Planned segment: F05. See the same feature heading in docs/demonstrations.md.

F06 Flashcard generation and review / UC06

Status: awaiting owner recording and upload. Video link: not supplied. Actual timestamp: not supplied. Planned segment: F06. See the same feature heading in docs/demonstrations.md.

F07 Quiz generation / UC07

Status: awaiting owner recording and upload. Video link: not supplied. Actual timestamp: not supplied. Planned segment: F07. See the same feature heading in docs/demonstrations.md.

F08 Automatic grading and explanations / UC08

Status: awaiting owner recording and upload. Video link: not supplied. Actual timestamp: not supplied. Planned segment: F08. See the same feature heading in docs/demonstrations.md.

F09 Weak-topic diagnosis and adaptive practice / UC09

Status: awaiting owner recording and upload. Video link: not supplied. Actual timestamp: not supplied. Planned segment: F09. See the same feature heading in docs/demonstrations.md.

F10 Progress and mastery analytics / UC10

Status: awaiting owner recording and upload. Video link: not supplied. Actual timestamp: not supplied. Planned segment: F10. See the same feature heading in docs/demonstrations.md.

F11 Study scheduling and reminders / UC11

Status: awaiting owner recording and upload. Video link: not supplied. Actual timestamp: not supplied. Planned segment: F11. See the same feature heading in docs/demonstrations.md.

F12 Orchestrated focused study session / UC12

Status: awaiting owner recording and upload. Video link: not supplied. Actual timestamp: not supplied. Planned segment: F12. See the same feature heading in docs/demonstrations.md.

References and final ownership

Stage 1 source: StudyForge_Stage1_Project_Design_Report.pdf (40 pages, unchanged). Stage 2 requirements: Project-Stage2-Instruction.pdf (5 pages, supplied locally; not republished without need).

Official API contract: Structured model outputs. JSON mode guarantees JSON shape only; application validators provide feature-level checks.

The student must review and understand the code, verify model output and billing, complete recordings and submit the repository through the course system. This report describes verified source and explicit limitations rather than claiming every submission gate has passed.